

RL-Fill: Timing-Aware Fill Insertion using Reinforcement Learning

Jinoh Cho, Seonghyeon Park, Jakang Lee, Sung-Yun Lee, Jinmo Ahn, and Seokhyeong Kang

Department of Electrical Engineering, Pohang University of Science and Technology (POSTECH)

Pohang, Republic of Korea

shkang@postech.ac.kr

ABSTRACT

We introduce RL-Fill, a novel reinforcement learning framework for timing-aware fill insertion. RL-Fill first generates a large number of fills in the empty spaces and then removes the timing-critical fills as determined by the policy network. Towards faster convergence and stability, our framework employs a two-phase training process. In the first phase, we train the policy with offline expert data using an imitation learning scheme. In the second phase, we further optimize the policy with online data using reinforcement learning. Moreover, we propose a new data augmentation method, LayoutMix, to ensure data-efficient training despite limited number of expert data. Our results demonstrate that RL-Fill is competitive to the commercial tool and outperforms the previous machine learning-based method in timing metrics while adhering density constraints.

ACM Reference Format:

Jinoh Cho, Seonghyeon Park, Jakang Lee, Sung-Yun Lee, Jinmo Ahn, and Seokhyeong Kang. 2024. RL-Fill: Timing-Aware Fill Insertion using Reinforcement Learning. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '24)*, October 27–31, 2024, New York, NY, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3676536.3676795>

1 INTRODUCTION

In the fabrication of silicon chips, chemical mechanical planarization (CMP) process plays a pivotal role in producing a perfectly flat foundation for stacking the next layer of circuit layout. The pursuit of a high degree of planarization is imperative, as deviations from flatness, known as dishing and erosion, can lead to poor yield. The post-CMP quality heavily depends on the uniform distribution of metal density across the layer surface. To address irregularities of metal density, inserting fills in the empty spaces between signal wires has become a common practice. However, this solution introduces a significant challenge: the interaction between the metal fills and signal wires can adversely affect the timing performance of the original circuit layout. Furthermore, this timing degradation issue increases as VLSI technology continues to scale down. Thus, there is a pressing need for sophisticated fill insertion algorithm considering not only uniformity of metal density but also timing degradation.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

ICCAD '24, October 27–31, 2024, New York, NY, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1077-3/24/10

<https://doi.org/10.1145/3676536.3676795>

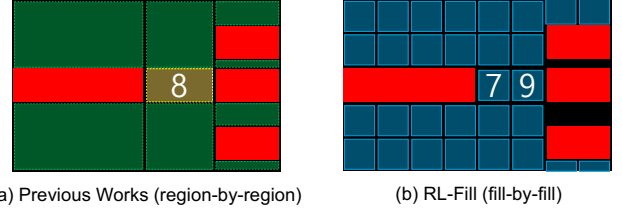


Figure 1: Comparison between fill-by-fill and region-by-region evaluation. RL-Fill can identify the timing cost in more fine-grained way. White numbers are the timing cost of fills or regions. Red and blue rectangles are timing critical wires and fills, respectively. Green rectangles are coarse evaluation units in previous works [1, 2].

Recent approaches [1, 2] have demonstrated promising outcomes by strategically allocating a smaller number of fills in timing-critical areas while placing a larger quantity in regions of lower criticality; however, these methods assess the timing criticality of fill regions in a coarse manner, which can lead to sub-optimal outcomes, as shown in Figure 1. Additionally, they rely on dated and simple heuristic guidelines from earlier research [3–6] for fill generation. These guidelines require updates when applied to other or recent technology nodes.

Reinforcement learning (RL) is gaining attention for its potential to serve as an alternative to traditional approaches across various domains in electronic design automation (EDA), such as placement [7–9], routing [10], gate sizing [11], and finding various parameters [12, 13]. However, learning policy using RL from scratch often requires long simulation times of commercial tools to calculate rewards and extensive iterations to converge. This tendency drives us to use offline expert data with imitation learning (IL) in conjunction with RL.

In this work, we propose a novel RL framework for timing-aware fill insertion, called RL-Fill. RL-Fill offers three major advantages. First, RL-Fill adopts a fill-by-fill evaluation rather than a region-by-region evaluation to achieve a more near-optimal fill pattern. Region-by-region evaluation can overlook the timing criticality of fills that, while located in the same evaluation unit (region), may have different timing criticalities. Second, RL-Fill learns an adaptive guideline (policy) tailored to the target technology node, avoiding the use of simple heuristic guidelines from dated research [3–6]. Third, RL-Fill improves the convergence of the policy network and reduces extensive exploration time by providing a near-optimal policy as a starting point, leveraging imitation learning with expert trajectories. We observe directly optimizing using RL causes unstable training and resulting in poor quality convergence within feasible time.

Furthermore, given that obtaining solutions from experts can be both time-consuming and costly, we introduce a new data augmentation technique called LayoutMix. This approach increase the volume of available data, thereby enable data-efficient training and leading to improvement of RL-Fill performance.

We summarize our contributions as follows:

- We address the traditional fill insertion problem using RL for the first time.
- RL-Fill learns an adaptive guideline (policy) tailored to target technology node, rather than relying on simple heuristic guidelines optimized for different technology nodes.
- RL-Fill evaluates timing criticality on a fill-by-fill basis instead of the traditional region-by-region approach, resulting in more sophisticated outcomes.
- We propose the sequential use of IL and RL to achieve faster convergence and improved stability.
- We introduce a novel data augmentation technique called LayoutMix, enabling data-efficient training and reducing the reliance on costly expert solutions.
- RL-Fill achieves state-of-the-art performance across various designs, comparable to the commercial tool and surpassing the previous machine learning method.

2 PRELIMINARIES

2.1 Problem Definition

The process of fill insertion can be considered a problem of finding fill allocations $\mathbf{a} = \{a_1, \dots, a_n\}$ that minimizing the interconnect delay objective $\mathcal{J}$ while evenly distributing density across sub-layouts (Gcells). This can be mathematically expressed as follows:

$$\mathbf{a}_{fill}^* = \arg \min_{\mathbf{a}} \mathcal{J}(\mathbf{a}; \mathbf{x}) \quad (1a)$$

$$\text{s.t. } 4D^{\min} \leq D_{i,j} + D_{i,j+1} + D_{i+1,j} + D_{i+1,j+1} \leq 4D^{\max},$$

$$\mu_{target}^{\min} \leq \mu \leq \mu_{target}^{\max}, \forall i, j. \quad (1b)$$

where $\mathbf{x}$ represents the target circuit layout, and $D_{i,j}$ denotes the metal density of the Gcell at the i -th row and j -th column when the entire layout $\mathbf{x}$ is segmented into multiple Gcells. The values $D^{\min}$ and $D^{\max}$ are set to control the density variation between neighboring Gcells, thereby regulating the target density variance after fill insertion is completed. Furthermore, $\mu_{target}^{\min}$ and $\mu_{target}^{\max}$ are established to control the acceptable range of mean density μ across the entire Gcells. Density constraints are verified across the entire layout $\mathbf{x}$ by employing a moving window of size $w \times w$ with a step size of $\frac{w}{2}$. To effectively manage these density constraints, we set the Gcell size to $\frac{w}{2} \times \frac{w}{2}$ in this paper similar to the previous work [1].

2.2 Related Works

Early studies [3, 4] establish foundational guidelines for fill insertion to reduce induced coupling capacitance. Former [3] introduces the eight guidelines for fill configurations such as avoiding specific high impact regions and latter [4] provides selection of a "good" fill pattern related to the number of rows and columns in fill pattern.

A separate line of researches address fill insertion problem by optimizing both the Min-Fill and Min-Var objectives simultaneously using various approaches [15, 16]. Early one [15] empirically shows advantages of Monte Carlo and heuristic strategies over Linear Programming (LP) methods. The latest one [16] presents a fully polynomial-time LP formulation to mitigate runtime issues of LP methods effectively.

In recent years, ICCAD has hosted two notable contests for fill insertion algorithms, one in 2014 [17] and another in 2018 [18]. The objective of the 2014 contest has been to minimize the overlap between fill regions and non-fill regions in neighboring layers, while also reducing the total number of fills. By 2018, the contest objectives have become more sophisticated, focusing on minimizing the capacitance of both critical and total nets within the designs. Notably, two state-of-the-art models [1, 2] have been presented using the 2018 contest benchmark. These models begin by assigning target densities to each sub-layout with consideration of timing criticality, then proceed to generate fills until the target densities are achieved. However, as mentioned earlier, these models [1, 2] may suffer from performance drops due to the coarse region-by-region timing criticality evaluation and the use of simple heuristic guidelines for fill generation from dated papers [3–6].

With the recent advancements in deep learning techniques, a recent approach [14] has pioneered the use of a Generative Adversarial Network (GAN), demonstrating promising results in directly translating post-route layout images into post-fill layout images using a timing-aware loss function. However, this approach faces challenges such as image blurriness, necessitating an extensive design rule violation correction stage. More significantly, it struggles to outperform commercial tools since the training datasets are sourced from these very tools. In contrast, RL-Fill does not rely on timing-aware post-fill layouts from commercial tools for training, thereby possessing the potential to surpass them, as demonstrated in this paper.

3 RL-FILL FRAMEWORKS

The RL-Fill framework operates through three distinct stages as illustrated in Figure 2: (1) All empty spaces within the layout are completely filled. This stage ensures there is enough flexibility for subsequent fill removals based on timing criticality. (Excessive Fill Generation), (2) After dividing the layout into multiple Gcells, as mentioned in Section 2.1, we allocate the expected number of fills to be removed based on the timing criticality of each Gcell. This approach enables the removal of a larger number of fills in high timing critical Gcells compared to those with lower timing criticality, while still adhering to density constraints. (Target Density Planning) (3) Finally, fills assumed to be timing critical are removed according to trained policy network until planned target densities are met. (Fill Removal)

3.1 Fill Removal as MDP

Reason for first-fill-next-remove. Starting with a zero-fill post-route layout and incrementally inserting fills one by one requires numerous fill insertion steps before an episode terminates. Upon analyzing post-fill layouts generated by commercial tools, we observe that most empty spaces are typically filled, while regions

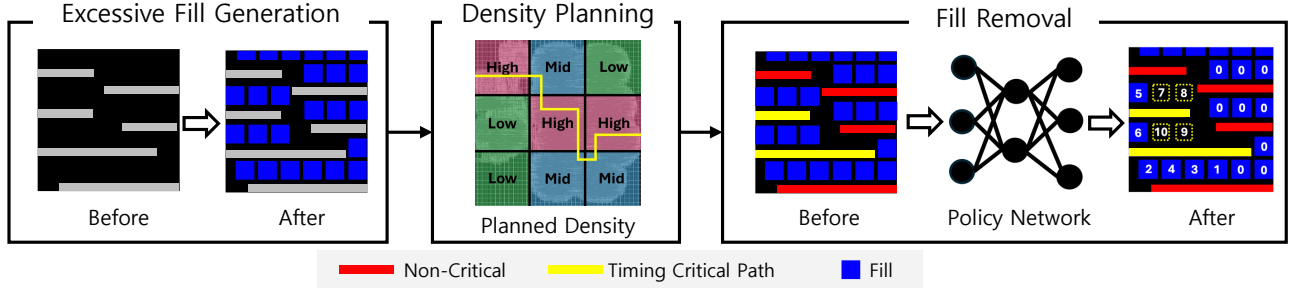


Figure 2: Overall RL-Fill Flow. We firstly completely fill the empty region, then plan target density of each Gcell. Gcells with timing critical path are set to have low target densities. Finally, we remove the timing critical fills using policy network until the target densities are met.

identified as timing critical are left unfilled. Building on this observation, our approach begins by initially filling all available spaces and then selectively removing fills based on timing criticality (fill-by-fill).

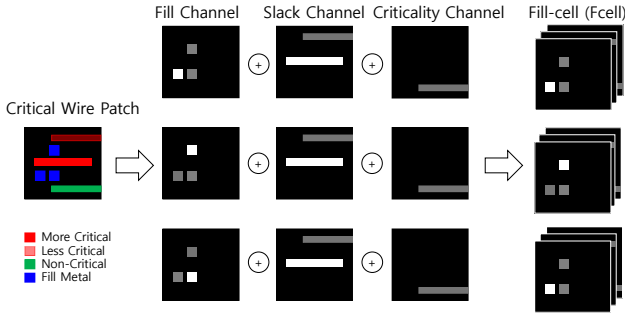


Figure 3: Illustration of Fcells generation process. First, we generate patches surrounding the critical wires. Then, we generate fill channel, slack channel and criticality channel images to obtain Fcell array.

State. The state of Gcell is represented by the collection of fill-cells (Fcell). To generate Fcells, we first sample all the $\delta\mu m \times \delta\mu m$ rectangular regions (critical wire patches) centered around timing critical wires which have negative slacks. Then, we have to generate three channel images including fill channel, slack channel and criticality channel as shown in Figure 3. First, we assign the value of 1 for the target fill and 0 for the other fills to generate fill channel images. Second, we assign normalized negative slack values for critical wires to generate slack channel images. Third, criticality channel images are generated by the whether the wires are critical or non-critical. Slack channel and criticality channel are shared across the Fcells with in same critical wire window. We also generate slack channel and criticality channel images for adjacent metal layer (top and bottom). By combining these images, we can construct final Fcells. For the input to the policy network, we concat the current Fcells and initial Fcells.

Finally, the state representation S_t is an array with dimensions $D \times C \times N \times N$, where D denotes the number of fills adjacent to timing critical wires within the Gcell, N is the width and height of the image representing the Fcell, and C represents the number of channels.

Action. The action a_t is represented as a D -dimensional vector, where each element indicates the priority score of each fill to be

removed. We obtain this vector by passing a set of Fcells to the policy network $\pi_\theta(s)$. To convert these scores with probability distribution, we apply a softmax function to these vector. Finally, we apply categorical sample function to the post-softmax outcomes to remove fills with the highest probability (or priority).

Reward. We maximize the capacitance decrement between the initial and final states using the following formulation. We only consider the capacitance values of wires that belong to the timing critical net within the Gcell. The total rewards are represented by the weighted sum of the capacitance decrements of all critical wires, with each wire's weight determined by its normalized negative slack.

$$\text{Reward} = \frac{1}{n} \sum_{i=0}^n w_i \frac{C_{init}^i - C_{final}^i}{C_{init}^i}, \quad (2)$$

where n is the number of signal metal, $w_i \in [0, 1]$ is the timing critical weight of i -th wire. C_{init}^i and C_{final}^i indicate the capacitance of wire i at the initial and final state, respectively. By defining the rewards as equation 2, we can expect more fills to be removed around timing critical wires. Consequently, we can effectively reduce the values of total negative slack (TNS) and worst negative slack (WNS).

3.2 Excessive Fill Generation

As mentioned in Section 3.1, to ensure that all empty regions are fully filled initially, we use the Innovus [22] tool in timing-off fill insertion mode with a high preferred density (target density plus extra density). This stage offers sufficient options for subsequent fill removals based on timing criticality.

3.3 Timing-Aware Fill (Density) Planning

This stage, similar to the previous research [1], involves determining how many fills need to be removed from each Gcell. Without this stage, the simplest approach would be to assign the same density to all Gcells. However, this could lead to sub-optimal outcomes from a timing perspective. Therefore, instead of assigning the same density to all Gcells, we allocate lower density to areas with a higher concentration of timing-critical wires and higher density to areas with fewer timing-critical wires, even at the cost of some loss in density variance. This can be mathematically formulated as follow:

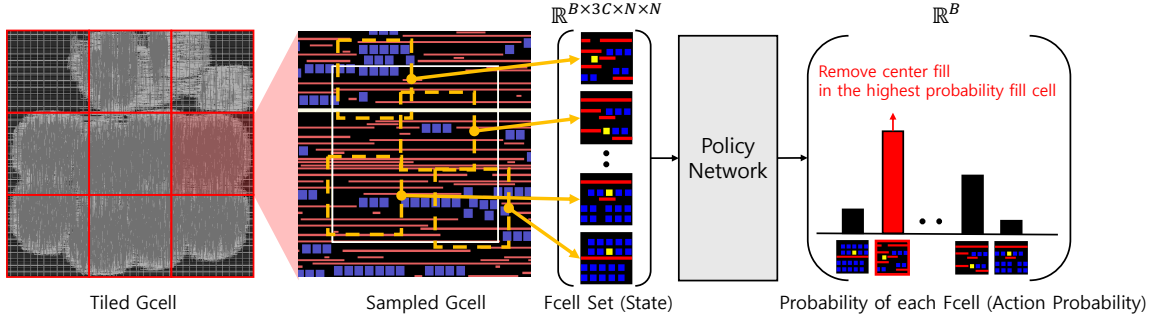


Figure 4: Fill Removal Stage Overview. Each Gcells comprises a collection of smaller regions surrounding each fill, called fill-cells (Fcells). A set of Fcells represent the current state of the Gcell. These Fcells are transform into probability by policy network. The fill (yellow-colored) in the Fcell with the highest probability will be removed. Each Gcell and Fcell are represented as a single metal layer image for simplicity.

$$\min \sum_{i,j} T_{i,j} D_{i,j} \quad (3a)$$

$$\text{s.t. } 4D^{\min} \leq D_{i,j} + D_{i,j+1} + D_{i+1,j} + D_{i+1,j+1} \leq 4D^{\max},$$

$$\mu_{\text{target}}^{\min} \leq \mu \leq \mu_{\text{target}}^{\max}, \forall i, j. \quad (3b)$$

Here, $D_{i,j}$, $D^{\min}$, $D^{\max}$, μ , $\mu_{\text{target}}^{\min}$, $\mu_{\text{target}}^{\max}$ are same with in Equation 1b. Notably, $T_{i,j}$ represents a coefficient that quantifies the timing criticality of Gcell in (i, j) after tiling. This is determined by assessing the extent to which each Gcell's area is occupied by the wires of the N critical and non-critical wires. This can be mathematically expressed as follows:

$$T_{i,j} = \begin{cases} \epsilon, & \text{if } a_{i,j} = 0, \\ (\sum_{n=0}^N \beta_n^{\text{cri}} \cdot a_{i,j,n}^{\text{cri}}) + \beta^{\text{non}} * a_{i,j}^{\text{non}}, & \text{else.} \end{cases} \quad (4)$$

Here, $a_{i,j}^{\text{cri}}$ denotes the portion of critical wire area within the Gcell $G_{i,j}$ located at (i, j) after Gcell tiling, while $a_{i,j}^{\text{non}}$ represents the portion of non-critical wire within $G_{i,j}$. The area portion of critical wire can be express the equation $a_{i,j}^{\text{cri}} = \sum_{n=0}^N a_{i,j,n}^{\text{cri}}$. The coefficient β_n^{cri} is largely assigned if the wire n belongs to the net with large negative slack.

By optimizing this objective function, we can strategically remove more fills from timing critical Gcells and fewer from non-critical ones. Specifically, the number of fills to be removed in each Gcell can be calculated by $\frac{(D_{i,j}^{\text{Full}} - D_{i,j}^{\text{Sol}})}{D_{\text{fill}}}$, where $D_{i,j}^{\text{Full}}$ is the density of the Gcell $G_{i,j}$ after excessive fill generation stage (see Section 3.2), $D_{i,j}^{\text{Sol}}$ is the target density of the Gcell $G_{i,j}$ determined by solving Equation 3a, and D_{fill} is the density of a single fill within a Gcell.

3.4 Fill Removal using Policy Network

After density planning stage, we should determine which fills should be removed. This is determined by the policy network $\pi_{\theta}(s)$.

Policy Network. Convolutional Neural Networks (CNNs) are commonly used in many computer vision tasks because they can effectively recognize patterns in images and extract important features. Similarly, our sampled Fcells have a complex arrangement, with various types of metals surrounding the fill removal candidates. Hence, we use a CNN architecture to efficiently extract spatial and

geometric information from these intricate metal patterns and to model the relationships between different metals and fills. Please refer the details in Section 4.3. By passing the sampled Fcells through this policy network, we can obtain priority scores for each fill removal candidate, as described in Figure 4.

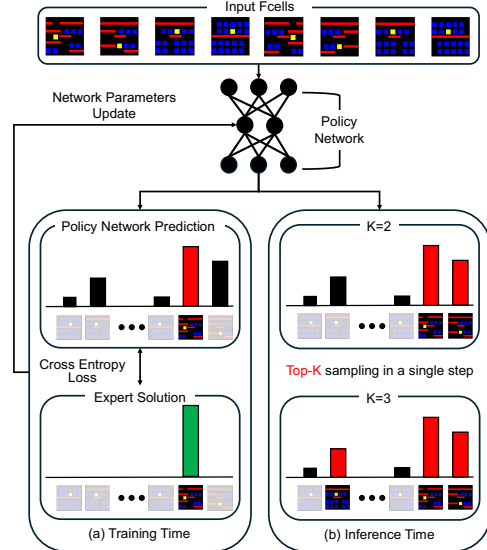


Figure 5: Single Step Inference. In inference time, we sample the target number of fills K in a single step by sampling top- K fills among FCells for the given initial Gcell. This dramatically reduces runtime.

Single Step Inference. During training, we remove one fill at a time from the current state, prioritizing removal based on the highest scores from the policy network, according to previously defined MDP structure in Section 3.1. In contrast, during inference, iterative processes are necessary for each Gcell due to the large number of fills generated in the excessive fill generation stage. These fills, often counting over hundreds or thousands, must be removed until satisfying density constraints. To accelerate inference, we do not remove fills one by one; instead, we remove the top- K fills simultaneously based on their removal priority scores from the policy network as shown in Figure 5. Here, K represents the predetermined number of fills to be removed from each Gcell during the density planning stage.

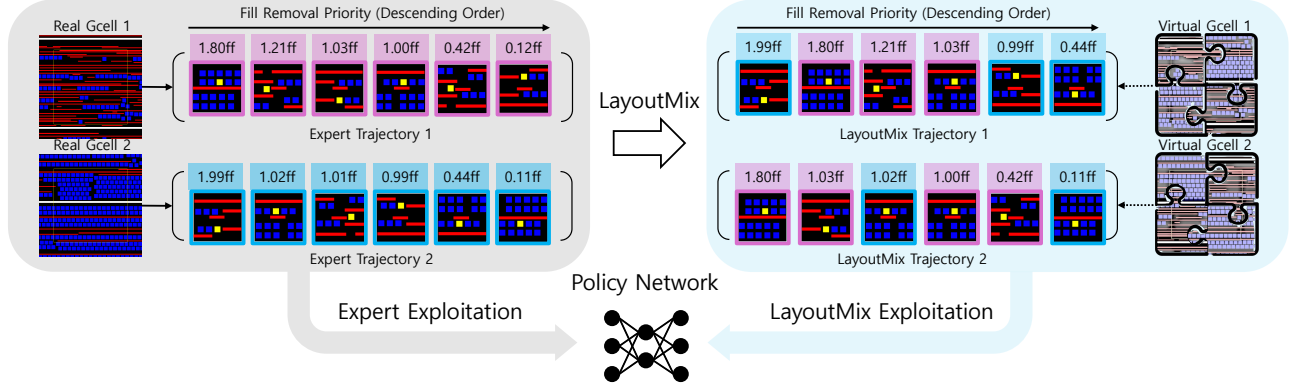


Figure 6: LayoutMix Exploitation. LayoutMix generates new expert-quality trajectories by mixing Fcells from different Gcells and ordering with their capacitance decrement. These trajectories can be thought as expert trajectories from virtually mixed Gcells in imagination.

3.5 Imitation Learning Phase

We first train the policy network with imitation learning framework. We propose to use two exploitation to training our policy network $\pi_\theta(s)$ as follow.

Expert Exploitation. Finding the optimal solution that maximizes capacitance reduction when removing n fills is computationally infeasible due to the inherent complexity. However, the optimal solution for removing a single fill can be determined within a feasible timeframe. This is achieved by individually removing each fill candidate, evaluating the capacitance values after each removal using Raphael [23], and selecting the fill that results in the greatest capacitance decrease.

Through this method, we can construct a sequence of state-action pairs, referred to as an expert trajectory. This trajectory is formally represented as $D_{\text{exp}} = \{(s_0, a_0), (s_1, a_1), \dots, (s_n, a_n)\}_{n=N}$, where N denotes the total number of fills. Although this expert trajectory does not represent the optimal solution, it serves as a near-optimal solution suitable for use in imitation learning. The process of leveraging this expert trajectory can be expressed as follows:

$$\mathcal{L}_{\text{Expert}} = -\mathbb{E}_{a,s \sim D_{\text{expert}}} [\log \pi_\theta(a|s)] \quad (5)$$

LayoutMix Exploitation. Obtaining expert solution trajectories is feasible, yet it requires substantial simulation time. To address this limitation, we propose LayoutMix, a methodology designed to augment existing expert trajectories and generate new, expert-quality trajectories without incurring the cost of additional simulations. As depicted in Figure 6, LayoutMix begins by randomly sampling sets of Fcells, independent of the original Gcells to which they belong. These sets, although not physically realized, represent Fcells from virtual Gcells, which are conceptualized by mixing real Gcells in an imagined space.

Each Fcell within these sets is pre-annotated with a cap decrement for expert exploitation. Leveraging these scores, we construct what we term a LayoutMix trajectory—a synthetic trajectory that maintains expert-level quality. These synthesized trajectories facilitate a data-efficient imitation learning framework, enabling the replication of expert behavior without the computational burden of

extensive simulations. The formulation of LayoutMix exploitation is as follows:

$$\mathcal{L}_{\text{LayoutMix}} = -\mathbb{E}_{a,s \sim D_{\text{LayoutMix}}} [\log \pi_\theta(a|s)] \quad (6)$$

Loss Function. To utilize both expert and LayoutMix trajectories, we can formulate the total loss function for training our policy network as follow:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{expert}} + \mathcal{L}_{\text{LayoutMix}} \quad (7)$$

Training algorithm. For the given a set of expert trajectories D_E , we can apply LayoutMix transformation to get a set of LayoutMix trajectories D_L . For every update step, we draw two batches of the state and action pairs from the D_E and D_L , respectively. Then, we update the policy network to follow the behaviors of experts and LayoutMix trajectories using the loss function defined in Equation 7. Details are described in Algorithm 1.

Algorithm 1: Imitation Learning Phase

Inputs : Max step T_{Max} , Expert trajectories set D_E , Learning rate α , Policy network π_θ
Output : Trained policy network π_θ

- 1 Initialize network parameters θ
- 2 Initialize $T_{\text{Current}} \leftarrow 0$
- 3 Initialize $D_L \leftarrow \text{LayoutMix}(D_E)$
- 4 **while** $T_{\text{Current}} < T_{\text{Max}}$ **do**
- 5 Sample (S_E, A_E) from D_E
- 6 Sample (S_L, A_L) from D_L
- 7 /* Sample state-action pairs from expert and LayoutMix trajectories */
- 8 $A_{EP} \leftarrow \pi_\theta(S_E)$
- 9 $A_{LP} \leftarrow \pi_\theta(S_L)$
- 10 /* Predict actions using policy network */
- 11 $L_E \leftarrow \text{CrossEntropyLoss}(A_{EP}, A_E)$
- 12 $L_L \leftarrow \text{CrossEntropyLoss}(A_{LP}, A_L)$
- 13 $L_{\text{Tot}} \leftarrow L_E + L_L$
- 14 $\theta \leftarrow \theta - \alpha \frac{\partial L_{\text{Tot}}}{\partial \theta}$
- 15 /* Update the parameters of policy network to follow expert and Layoutmix trajectories */
- 16 $T_{\text{Current}} = T_{\text{Current}} + 1$
- 17 **return** π_θ

3.6 Reinforcement Learning Phase

Towards finding optimal solution using our policy network, we further fine-tune the policy network using the UDRL algorithm [21]. The MDP structure and reward function described in Section 3.1 are employed in this process. For a comprehensive explanation of the UDRL algorithm, please refer to the original paper [21].

4 EXPERIMENTAL RESULTS

All the experiments are conducted in on a CentOS Linux 7 server equipped with NVIDIA GeForce RTX 3090 GPU, Intel(R) Xeon(R) Gold 6226R 2.90GHz CPU and 472G RAM.

4.1 Dataset

Experiments are conducted using the open source designs from OpenCores [28]. The OpenCores benchmarks are firstly synthesized through Design Compiler [25], then the outcomes are placed and route using Innovus [22] with industrial 22nm technology node, sequentially. We set a 50% utilization rate for all designs, except for keccak, which is set at 30% to allow ample space for fill insertion. The aspect ratio is set to 1.0 for all designs. We use total 13 designs as described in Table 1. These designs contain various number of signal wires from 5K to 141K. In this work, we only utilize metal wires from metal layer 3 to 5 to insert fills in metal layer 4.

Table 1: Design Statistics

Designs	Clock (ps)	Die Size ($\mu\text{m} \times \mu\text{m}$)	Wire Count
<i>aes_cipher_top</i>	1000	100.22 $\times$ 99.54	12197
<i>eth_top</i>	550	224.10 $\times$ 223.65	34210
<i>jpeg_encoder</i>	580	221.57 $\times$ 210.42	40209
<i>keccak</i>	1950	170.21 $\times$ 169.47	28743
<i>mc_top</i>	1000	76.14 $\times$ 75.60	4604
<i>mpeg2_top</i>	1100	141.70 $\times$ 141.12	13496
<i>nova</i>	3500	410.08 $\times$ 409.50	141910
<i>pci_bridge32</i>	600	127.01 $\times$ 126.63	11985
<i>point_scalar_mult</i>	800	219.02 $\times$ 217.98	44841
<i>tate_pairing</i>	1050	465.59 $\times$ 464.94	190110
<i>tv80s</i>	1200	65.02 $\times$ 64.89	3994
<i>vga_enh_top</i>	500	280.04 $\times$ 279.72	50193
<i>wb_conmax_top</i>	1000	131.652 $\times$ 131.04	19604

Train Data. We randomly sample 100 Gcells, each size of $5\mu\text{m} \times 5\mu\text{m}$, from 13 different designs, yielding a total of 1,300 Gcells. For zero-shot evaluation, we trained our model on data from 12 designs, excluding the design under assessment. For instance, to assess RL-Fill on *keccak*, it is trained using datasets derived from the other 12 designs.

Test Data. For testing, each design is divided into $25\mu\text{m} \times 25\mu\text{m}$ Gcells, using a $50\mu\text{m} \times 50\mu\text{m}$ density window with a $25\mu\text{m}$ step size, as explained in Section 2.1. To highlight again, we exclude the test design from training data to evaluate the generalization of our RL-Fill policy network on unseen designs (zero-shot).

4.2 Baseline

Commerical Tool. Innovus [22] features two modes for timing-aware fill insertion: "On" and "STA." In the "On" mode, nets are prioritized based on timing costs, which are assigned in descending

order: clock nets, signal nets, and then power and ground nets. Fills are then inserted starting with the area near the lowest timing cost net. The "STA" mode utilizes static timing analysis results to allocate costs to each net, with metal fills added in a manner similar to the "On" mode.

GAN-Dummy Fill. Our study includes a comparison with GAN-based fill insertion framework [14], which employ a generative model equipped with a timing-aware loss function to generate fill patterns.

4.3 Implementation and Training Details

We implement the model using the Pytorch [19] library in Python. The architecture consists of three convolutional blocks and three fully connected blocks, as described in Table 2.

Imitation Learning Phase. We employ the Adam [20] optimizer with hyper-parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The learning rate and the batch size is set at 0.0002 and 128, respectively. The models are trained over 200K iterations.

Reinforcement Learning Phase. We use the same optimizer, learning rate, and batch size as in the imitation learning phase. The number of warm-up episodes is set to 100, and the replay buffer size is configured to 400. Additionally, 25 exploration episodes are added to the replay buffer at each iteration.

Table 2: Policy Network Architecture.

Block Name	Component
Conv Block 1	Conv(IC=9, OC=32, KS=3, S=2), ReLU
Conv Block 2	Conv(IC=32, OC=64, KS=3, S=2), ReLU
Conv Block 3	Conv(IC=64, OC=64, KS=3, S=2), ReLU
FC Block 1	FC (ID=9*9*64, OD=512, KS=3, S=2), ReLU
FC Block 2	FC (ID=512, OD=256, KS=3, S=2), ReLU
FC Block 3	FC (ID=256, OD=1, KS=3, S=2)

* IC and OC refers to input and output channel of convolution layer.

* KS and S refers to kernel size and stride of convolution layer.

* ID and OD indicate the input and output dimension of fully connected layer.

4.4 Comparison with Commerical Tool

After obtaining final outcomes with RL-Fill and Innovus, we compare the TNS and WNS using the StarRC [27] and Prime Time [26] tools. Runtime does not include any data preprocessing or tool loading time. We only report runtime of core fill insertion algorithms in Table 3. RL-Fill needs extra minutes of preprocessing time to preparing Fcells (5~10 minutes depending on the designs). For the density constraints, we set the mean density to be between 27 and 28, with the variance required to be below 1 for all algorithms. Additionally, all metal fill shapes are set to be square, with an edge length of $0.25\mu\text{m}$ and a minimum spacing of $0.05\mu\text{m}$.

As shown in Table 3, RL-Fill significantly outperforms Innovus in 8 out of 13 designs for TNS and in 9 out of 13 for WNS, reducing timing degradation by about 16% in TNS and 14% in WNS on average. These improvements are due to RL-Fill's fill-by-fill evaluation and its learned timing-aware fill removal guideline (policy). In contrast, conventional methods use coarse region-by-region evaluations with simplistic heuristic guidelines, which are inadequate for handling the complex patterns found in modern circuit designs,

Table 3: Timing metrics comparison with Innovus.

Designs	TNS (ps)				WNS (ps)				Runtime (s)			
	w/o fill	On	STA	RL-Fill	w/o fill	On	STA	RL-Fill	w/o fill	On	STA	RL-Fill
<i>aes_cipher_top</i>	-307.52	-344.15	-343.67	-359.92	-50.07	-51.79	-51.64	-52.95	-	0.63	11.07	12.77
<i>eth_top</i>	-111.05	-118.10	-119.67	-116.85	-74.30	-79.35	-79.17	-79.84	-	1.94	30.07	33.89
<i>jpeg_encoder</i>	-95.81	-124.39	-125.55	-119.64	-21.81	-25.31	-24.51	-22.90	-	2.14	24.77	28.12
<i>keccak</i>	-49.26	-53.97	-53.56	-52.47	-44.01	-45.20	-44.99	-44.72	-	1.88	31.05	24.85
<i>mc_top</i>	-1178.95	-1238.92	-1237.95	-1192.54	-56.84	-59.03	-58.54	-58.45	-	0.35	6.39	5.63
<i>mpeg2_top</i>	-1219.55	-1394.18	-1428.41	-1402.75	-86.05	-98.20	-98.85	-97.38	-	0.81	14.89	12.41
<i>nova</i>	-277.41	-306.46	-303.31	-301.25	-186.10	-194.21	-195.89	-192.93	-	14.70	110.78	172.40
<i>pci_bridge32</i>	-664.61	-702.14	-704.85	-708.54	-72.47	-74.08	-73.73	-73.62	-	0.67	12.83	10.83
<i>point_scalar_mult</i>	-162.21	-186.37	-184.99	-180.89	-47.50	-52.31	-51.69	-51.45	-	2.85	36.38	37.59
<i>tate_pairing</i>	-236.21	-948.89	-964.58	-967.74	-14.72	-21.02	-22.78	-22.34	-	17.52	136.50	245.23
<i>tv80s</i>	-152.96	-157.20	-158.02	-157.98	-39.54	-40.17	-40.20	-40.07	-	0.24	7.06	8.99
<i>vga_enh_top</i>	-41.00	-46.76	-45.97	-44.09	-29.18	-30.60	-30.96	-29.87	-	6.58	49.93	60.11
<i>wb_conmax_top</i>	-2464.90	-2640.80	-2628.56	-2573.28	-42.27	-44.20	-44.33	-43.45	-	1.11	20.14	24.44
Norm. Avg	-	0.99	1.00	0.84	-	1.05	1.00	0.86	-	0.08	1.00	1.09

* There are two timing-aware fill insertion modes (On, STA) in Innovus. w/o fill denotes post-route layout without any fills.

* The norm value of TNS and WNS are calculated with dividing $Value_{model} - Value_{w/o\ fill}$ with $Value_{STA} - Value_{w/o\ fill}$.

* The norm value of runtime is calculated with dividing $Value_{model}$ with $Value_{STA}$.

* The best results are highlighted in **bold**.

where many intertwined nets require a more precise approach for optimal fill pattern.

Even excluding preprocessing time, the runtime of the core RL-Fill framework is still longer than that of Innovus. This discrepancy arises because the RL-Fill framework applies the fill removal stage sequentially to each Gcell, one at a time, rather than leveraging the policy network to remove fills across all Gcells simultaneously. Therefore, adopting multi-processing in the future could significantly reduce the runtime. It is also important to note that, unlike other RL frameworks used in EDA, RL-Fill does not require any training on the target circuit. This greatly shortens the time from setup to application, as it directly infers on the target design.

4.5 Comparison with ML-based work

We leverage the original source code of GAN-Dummy Fill and conducted comparative evaluations against five designs, as illustrated in Table 4. The results demonstrate that RL-Fill significantly outperforms GAN-Dummy Fill across all benchmarks in terms of all timing and runtime metrics. Furthermore, GAN-Dummy Fill fails to surpass the performance of existing commercial tools. This limitation is likely attributed to the method employed in collecting the training dataset. Since GAN-Dummy Fill tends to generate patterns that mimic those observed during training, the quality of the generated patterns typically are bounded to that of the commercial tool outcomes.

Additionally, GAN-Dummy Fill often inherits issues related to blurriness and artifacts from the GAN architecture, necessitating a post-processing step. These issues are critical in EDA tools, as they can cause design rule violations. The required post-processing significantly slows down the generation of the final fill patterns, impacting overall efficiency.

Table 4: Comparison with GAN-Dummy Fill.

Designs	TNS (ps)	WNS (ps)	Runtime (s)
<i>keccak</i>	-54.59 (-2.12)	-46.45 (-1.73)	327.93 (+303.08)
<i>vga_enh_top</i>	-46.69 (-2.60)	-32.10 (-2.23)	850.54 (+790.43)
<i>eth_top</i>	-120.21 (-3.36)	-80.22 (-0.38)	616.25 (+582.36)
<i>point_scalar_mult</i>	-186.50 (-5.61)	-55.49 (-4.04)	522.04 (+484.45)
<i>nova</i>	-310.55 (-9.30)	-198.10 (-5.17)	1851.69 (+1679.29)

* Black values indicate results of GAN-Dummy Fill.

* Red values indicate gap ($Value_{GAN} - Value_{RL-Fill}$) between RL-Fill and GAN-Dummy Fill.

4.6 Ablation studies

Effectiveness of Single Step Inference. We verify the efficacy of Single Step Inference (SSI) techniques for fill removal by comparing two approaches: conventional RL-Fill, which removes one fill at a time, and RL-Fill with Single Step Inference, which samples the target number K of fills in Gcells in a single step by selecting the top- K Fcells. This comparison is conducted across three distinct designs. In Table 5, utilizing Single Step Inference to directly sample top- K Fcells does not result in a significant degradation in TNS and WNS, while reducing the runtime by an average of 219 times.

Table 5: Ablation of Single Step Inference Result.

	<i>mc_top</i>			<i>mpeg2_top</i>			<i>tv80s</i>		
	TNS	WNS	Runtime	TNS	WNS	Runtime	TNS	WNS	Runtime
w/ SSI	-1192.54	-57.45	5.63	-1402.75	-97.38	12.41	-157.97	-40.06	6.11
w/o SSI	-1191.53	-57.33	1876.16	-1403.18	-97.36	3037.37	-156.12	-39.33	492.25

* Units of TNS and WNS are ps. Unit of runtime is s.

* SSI refers to Single Step Inference.

Effectiveness of LayoutMix Exploitation. To evaluate the effectiveness of LayoutMix Exploitation, we conduct experiments across various setups with different numbers of expert trajectories (150, 300, 600, and 1200). Our findings, as illustrated in Figure 7, indicate that LayoutMix Exploitation consistently enhances the total reward compared to using expert exploitation alone. Notably, the performance gaps stand out at lower numbers of expert solutions, such as 150 and 300. This pattern suggests that LayoutMix is

particularly beneficial in environments where only limited expert trajectories are available.

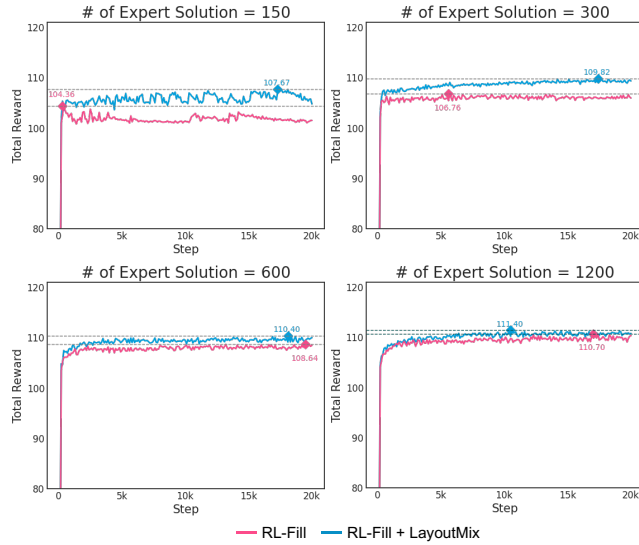


Figure 7: Ablation of LayoutMix Exploitation. The LayoutMix Exploitation robustly improves the performance of RL-Fill, irrespective of the number of expert solutions.

5 CONCLUSION

In this study, we introduce RL-Fill, a novel reinforcement learning framework for timing-aware fill insertion. RL-Fill offers three major advantages: (1) fill-by-fill evaluation instead of region-by-region evaluation, (2) learning-based adaptive guidelines rather than fixed ones optimized for other technology nodes, and (3) enhanced stability and convergence through the incorporation of imitation learning. Moreover, we propose a new data augmentation technique to efficiently leverage expert trajectories, which is beneficial in EDA, where obtaining expert solutions often requires intensive simulations or specialized expertise. Additionally, RL-Fill can generate fill patterns directly without training on the target design, significantly reducing design-to-market time. In experiment, we show that RL-Fill performs comparably to commercial tools and surpasses the machine learning baseline. Future work will focus on considering net-wise capacitance value rather than that of the wires, as well as exploring scalability to real-world designs.

ACKNOWLEDGMENTS

This work was partly supported by Institute for Information & communications Technology Promotion (IITP) grant funded by the Korea government (MSIP) (No. RS-2023-00222085 : Development of memory module and memory compiler for non-volatile PIM and No. RS-2023-00222609 : Development of thermal modelling and heat-spreading architecture for heterogeneous integrating package of AI chiplets) and Samsung Electronics Co., Ltd.

REFERENCES

- [1] Bentian Jiang, Xiaopeng Zhang, Ran Chen, Gengjie Chen, Peishan Tu, Wei Li, Evangeline F.Y. Young, and Bei Yu, "FIT: Fill Insertion Considering Timing", *Proc. DAC*, 2019.
- [2] Sheng-Jung Yu, Chen-Chien Kao, Chia-Han Huang, and Iris Hui-Ru Jiang, "Equivalent Capacitance Guided Dummy Fill Insertion for Timing and Manufacturability", *Proc. DAC*, 2020.
- [3] Andrew B. Kahng, K. Samadi, and P. Sharma, "Study of floating fill impact on interconnect capacitance", *Proc. ISQED*, 2006.
- [4] Lei He, Andrew B. Kahng, King Ho Tam, and Jinjun Xiong, "Variability-driven considerations in the design of integrated-circuit global interconnects", *Proc. VLSI*, 2004.
- [5] Andrew B. Kahng, and Rasit Onur Topaloglu, "A DOE Set for Normalization-Based Extraction of Fill Impact on Capacitances", *Proc. ISQED*, 2007.
- [6] Atsushi Kurokawa, Toshiaki Kanamoto, Tetsuya Ibe, Akira Kasebe, Chang Wei Fong, Tetsuro Kage, Yasuaki Inoue and Hiroo Masuda, "Dummy filling methods for reducing interconnect capacitance and number of fills", *Proc. ISQED*, 2005.
- [7] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Sungmin Bae, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Anand Babu, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean, "Chip Placement with Deep Reinforcement Learning", *arXiv:2004.10746*, 2020.
- [8] Uday Mallappa, Sreedhar Pratty, and David Brown, "RLPlace: deep RL guided heuristics for detailed placement optimization", *Proc. DATE*, 2022.
- [9] Sung-Yun Lee, Seonghyeon Park, Daeyeon Kim, Minjae Kim, Tuyen P. Le, and Seokhyeong Kang, "RL-Legalizer: Reinforcement Learning-based Cell Priority Optimization in Mixed-Height Standard Cell Legalization", *Proc. DATE*, 2023.
- [10] Yibo Lin, Tong Qu, Zongqing Lu, Yajuan Su, and Yayi Wei, "Asynchronous Reinforcement Learning Framework and Knowledge Transfer for Net-Order Exploration in Detailed Routing", *IEEE Trans. on CAD*, vol. 41, no. 9, pp. 3132-3142, 2022.
- [11] Yi-Chen Lu, Siddhartha Nath, Vishal Khandelwal, and Sung Kyu Lim, "RL-Sizer: VLSI Gate Sizing for Timing Optimization using Deep Reinforcement Learning", *Proc. DAC*, 2021.
- [12] Anthony Agnesina, Kyungwook Chang, and Sung Kyu Lim, "VLSI Placement Parameter Optimization using Deep Reinforcement Learning", *Proc. ICCAD*, 2020.
- [13] Sung-Yun Lee, Kyungjun Min, and Seokhyeong Kang, "CTRL-B: Back-End-Of-Line Configuration Pathfinding using Cross-Technology Transferable Reinforcement Learning", *Proc. DATE*, 2024.
- [14] Myong Kong, Daeyeon Kim, Minhyuk Kweon, and Seokhyeong Kang, "GAN-Dummy Fill: Timing-aware Dummy Fill Method using GAN", *Proc. GLSVLSI*, 2022.
- [15] Yu Chen, Andrew B. Kahng, Gabriel Robins, and Alexander Zelikovskiy, "Closing the smoothness and uniformity gap in area fill synthesis", *Proc. ISPD*, 2002.
- [16] Chunyang Feng, Hai Zhou, Changhao Yan, Jun Tao, and Xuan Zeng, "Efficient Approximation Algorithms for Chemical Mechanical Polishing Dummy Fill", *IEEE Trans. on CAD*, vol. 30, no. 3, pp. 402-415, 2011.
- [17] Rasit O. Topaloglu, "ICCAD-2014 CAD contest in Design for Manufacturability Flow for Advanced Semiconductor Nodes and Benchmark Suite", *Proc. ICCAD*, 2014.
- [18] Yang Bo and Sriraaman Sridharan, "ICCAD-2018 CAD contest in Timing-Aware Fill Insertion", *Proc. ICCAD*, 2018.
- [19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai and Soumith Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", *Proc. NeurIPS*, 2019.
- [20] Kingma, Diederik P and Ba, Jimmy, "Adam: A method for stochastic optimization", *Proc. ICLR*, 2015.
- [21] Rupesh Kumar Srivastava, Pranav Shyam, Filipe Mutz, Wojciech Jaśkowski and Jürgen Schmidhuber, "Training Agents using Upside-Down Reinforcement Learning", *NeurIPS Deep RL workshop*, 2019.
- [22] Cadence Inc, "Innovus 2021", in *Software*, 2021.
- [23] Synopsys Inc, "Raphael 2021", in *Software*, 2021.
- [24] Synopsys Inc, "IC Compiler 2020", in *Software*, 2020.
- [25] Synopsys Inc, "Design Compiler 2018", in *Software*, 2018.
- [26] Synopsys Inc, "Prime Time 2022", in *Software*, 2022.
- [27] Synopsys Inc, "StarRC 2021", in *Software*, 2021.
- [28] OpenCores: "Open Source IP-Cores," <http://www.opencores.org>.